Desafio Técnico: Sistema de Cadastro de Clientes

Documento de requisitos · versão 2.0 (parte 2 a partir da seção 19) · Oct 2, 2026 · @Urtium

1. Objetivo e formato

Uma dupla de desenvolvedores júnior constrói, em 10 dias úteis, um cadastro de clientes completo: API REST e interface web. O trabalho segue o fluxo de um time real, e a avaliação mede tanto o produto quanto o caminho até ele (planejamento, Gitflow, revisão de código e releases).

- **Equipe:** os dois devs trabalham no mesmo repositório GitHub; os avaliadores entram como colaboradores.
- **Prazo sugerido:** 10 dias úteis até a release v1.0.0, mais 1 dia útil para um hotfix pedido pelos avaliadores (seção 13). Ajustável.
- **Entrega:** tag v1.0.0 na branch main e apresentação de 30 minutos (seção 16).
- **Dúvidas:** abertas como issue no repositório, com a label `question`, para ficarem registradas junto do trabalho.
- **Idioma:** código, commits e nomes de branch em inglês; interface, README e textos da documentação da API em português.

2. Requisitos obrigatórios

Os 15 itens abaixo são o mínimo exigido; faltar qualquer um conta como entrega incompleta. Cada um tem critérios de aceite na seção indicada.

#	Requisito	Detalhado em
1	Node.js com TypeScript	Seções 5 e 10
2	Prisma como ORM	Seções 6 e 11
3	Técnicas de tipagem forte	Seção 10
4	Ao menos uma lib type safe na API (recomendada: Zod com fastify-type-provider-zod)	Seções 5 e 8

#	Requisito	Detalhado em
5	API REST	Seção 8
6	Interface em React, Vite e Tailwind CSS: listagem, wizard de cadastro, edição, soft delete e exclusão definitiva	Seção 9
7	Interface funcionando no celular	Seção 9
8	PostgreSQL leve com dados que sobrevivem a reinícios	Seção 11
9	Migrations versionadas	Seção 11
10	Gitflow	Seção 13
11	Clean code	Seção 14
12	Sistema conteinerizado	Seção 12
13	Inicialização com um único comando, via shell script	Seção 12
14	Tela para restaurar clientes excluídos (lixeira)	Seção 9
15	Documentação da API	Seção 8

3. Escopo

O sistema cobre o ciclo de vida inteiro de um cliente: cadastro, edição, lixeira, restauração e exclusão definitiva. Não há login.

Dentro do escopo

- API REST de clientes: listar, detalhar, cadastrar, editar, mover para a lixeira, restaurar e excluir definitivamente.
- Interface web responsiva para todos esses fluxos.
- Banco PostgreSQL com migrations e dados persistentes.
- Containers para banco, API e interface, iniciados por um único script.
- Documentação interativa da API.

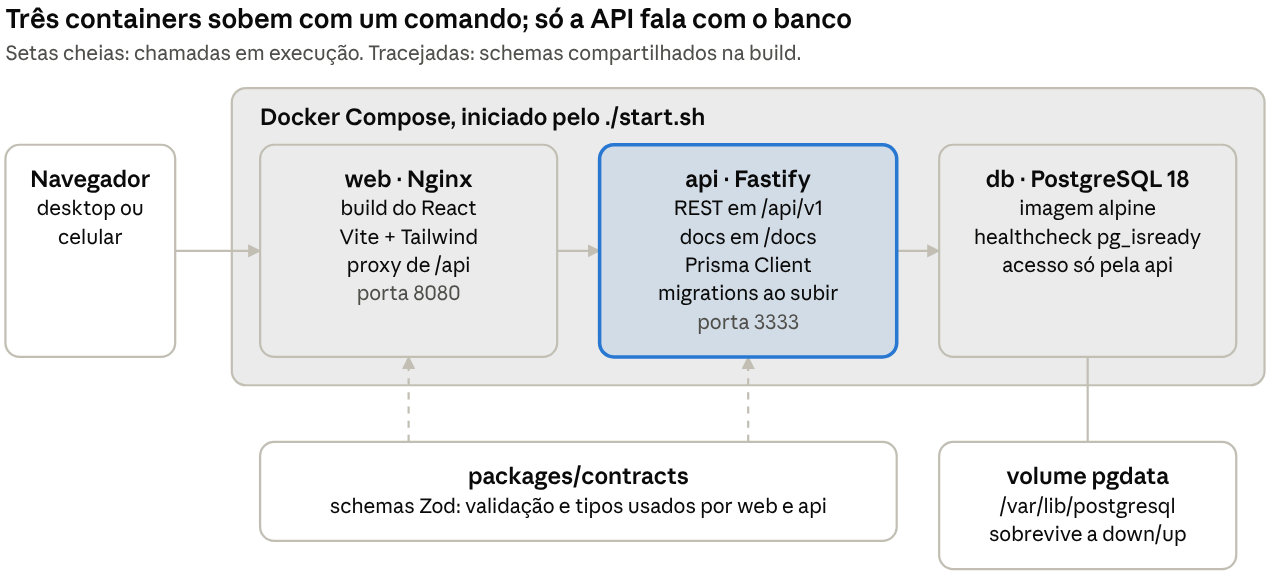
Fora do escopo

- Login, perfis e permissões.

- Deploy em nuvem.
- Vários endereços por cliente, anexos e histórico de alterações.
- Outros idiomas na interface.

4. Arquitetura e estrutura do repositório

O sistema roda em três containers (interface, API e banco), e o código vive em um monorepo com um pacote de contratos compartilhado.



arquitetura · 3 containers, 1 pacote compartilhado, 1 volume

O navegador só conversa com o Nginx, que repassa `/api` para a API; só a API acessa o banco. O pacote de contratos não é um container: é código importado por web e api na build.

Estrutura do repositório (npm workspaces):

```
cadastro-clientes/
  apps/
    api/            Fastify + Prisma
    web/            React + Vite + Tailwind
  packages/
    contracts/      schemas Zod e tipos compartilhados
  docker-compose.yml
  start.sh
  .env.example
  README.md
```

5. Stack

A lib type safe recomendada é o Zod integrado ao Fastify via fastify-type-provider-zod. O mesmo schema valida a requisição, gera os tipos da rota e alimenta o OpenAPI, então código, tipos e documentação não se desencontram.

Camada	Obrigatório	Recomendado
Linguagem	Node.js LTS e TypeScript em modo estrito	Versão do Node fixada em `.nvmrc` e no Dockerfile
API	REST, Zod e uma lib de rotas type safe	Fastify com fastify-type-provider-zod
ORM e migrations	Prisma e Prisma Migrate	Prisma Studio para inspeção local
Banco	PostgreSQL na imagem oficial alpine	Tag de versão fixa, nunca `latest`
Documentação da API	OpenAPI 3 gerado a partir dos schemas	@fastify/swagger com Swagger UI ou Scalar em `/docs`
Interface	React, Vite e Tailwind CSS	React Router, TanStack Query e React Hook Form
Formulários	Validação com os schemas Zod compartilhados	@hookform/resolvers (zodResolver)
Componentes	Diálogos e menus acessíveis por teclado	Radix UI ou Headless UI

Camada	Obrigatório	Recomendado
Infra	Docker, Docker Compose e `start.sh`	Nginx servindo o build da interface e fazendo proxy de `/api`
Qualidade	Lint e formatação sem erros nem avisos	typescript-eslint (strict-type-checked) com Prettier, ou Biome
Monorepo	Um repositório com API, interface e contratos	npm workspaces

Alternativa aceita para a API: Hono com @hono/zod-openapi, desde que entregue as mesmas garantias. tRPC não atende, porque não é REST.

6. Modelo de dados

São duas tabelas: `customers` (cliente PF ou PJ) e `addresses` (um endereço por cliente, apagado junto na exclusão definitiva). No banco, tabelas e colunas ficam em snake_case (com `@@map` e `@map`); no código, em camelCase.

Customer

Campo	Tipo	Obrigatório	Regra
id	UUID	Sim	Gerado no cadastro
personType	enum INDIVIDUAL ou COMPANY	Sim	PF ou PJ; não muda depois do cadastro
document	texto	Sim	CPF (11) ou CNPJ (14) sem máscara; único em toda a base; não muda
name	texto, 3 a 150 caracteres	Sim	Nome completo (PF) ou razão social (PJ)
tradeName	texto, até 150 caracteres	Só PJ, opcional	Nome fantasia
birthDate	data	Só PF, obrigatório	Não pode ser futura nem anterior a 1900
email	texto, até 254 caracteres	Sim	Formato válido, salvo em minúsculas

Campo	Tipo	Obrigatório	Regra
phone	texto	Sim	10 ou 11 dígitos, com DDD
createdAt	data e hora	Sim	Preenchido pelo banco
updatedAt	data e hora	Sim	Atualizado a cada alteração
deletedAt	data e hora	Não	Preenchido no soft delete; nulo enquanto ativo

Address

Campo	Tipo	Obrigatório	Regra
id	UUID	Sim	Gerado no cadastro
customerId	UUID	Sim	Chave estrangeira única para customers, com exclusão em cascata
zipCode	texto	Sim	CEP com 8 dígitos
street	texto	Sim	Logradouro, até 150 caracteres
number	texto	Sim	Até 10 caracteres; aceita "S/N"
complement	texto	Não	Até 100 caracteres
district	texto	Sim	Bairro, até 100 caracteres
city	texto	Sim	Até 100 caracteres
state	texto, 2 caracteres	Sim	Uma das 27 UFs

Índices obrigatórios: único em `customers.document`, comum em `customers.deleted_at` e único em `addresses.customer_id`.

No banco, PF e PJ dividem a mesma tabela com colunas opcionais. Na API e na interface, o cliente é uma união discriminada por `personType`, e a conversão entre os dois formatos fica em um mapper (seção 10).

7. Regras de negócio

O documento (CPF ou CNPJ) identifica o cliente: é único em toda a base, validado pelos dígitos verificadores e nunca muda. As demais regras definem o ciclo ativo, lixeira e exclusão definitiva.

Código	Regra	Resposta da API
RN01	Documento único em toda a base, inclusive na lixeira, garantido por constraint no banco (não só por validação na aplicação)	Ativo: 409 `customer-document-conflict`. Na lixeira: 409 `customer-in-trash`, com o `customerId`
RN02	CPF com 11 dígitos, dígitos verificadores corretos e sem sequência repetida (como 111.111.111-11)	400 no campo `document`
RN03	CNPJ com 14 caracteres e dígitos verificadores corretos, aceitando o formato alfanumérico (detalhes abaixo)	400 no campo `document`
RN04	`personType` e `document` não mudam depois do cadastro	400 se vierem na edição
RN05	Campo exclusivo de um tipo não vale para o outro: `birthDate` só PF, `tradeName` só PJ	400 no campo enviado
RN06	Documento, telefone e CEP chegam com ou sem máscara e são salvos sem pontuação; textos passam por trim; e-mail vai para minúsculas e letras do CNPJ para maiúsculas	Respostas trazem sempre o valor normalizado
RN07	Soft delete preenche `deletedAt`: o cliente sai da listagem, não pode ser editado e aparece na lixeira	Editar ou mover para a lixeira de novo: 404
RN08	Restaurar limpa `deletedAt` e devolve o cliente à listagem com os dados originais	Cliente fora da lixeira: 409

Código	Regra	Resposta da API
RN09	Exclusão definitiva apaga cliente e endereço, esteja ativo ou na lixeira, e não pode ser desfeita	Cliente inexistente: 404
RN10	Ordem padrão: listagem por `createdAt` decrescente; lixeira por `deletedAt` decrescente	Alterável por `sortBy` e `sortOrder`
RN11	Datas trafegam em ISO 8601 (UTC para data e hora, AAAA-MM-DD para `birthDate`) e aparecem como dd/mm/aaaa na interface	Formato inválido: 400

CNPJ alfanumérico. A partir de julho de 2026, novos CNPJs podem ter letras (A a Z) nas 12 primeiras posições; os 2 dígitos verificadores continuam numéricos e a máscara não muda. O cálculo segue o módulo 11, com cada caractere valendo seu código ASCII menos 48 (A = 17, B = 18 e assim por diante). Fonte: perguntas e respostas da Receita Federal. CNPJs fictícios para teste podem ser gerados no Simulador Nacional de CNPJ.

Casos de teste mínimos dos validadores

Valor	Resultado esperado
529.982.247-25	CPF válido
529.982.247-26	CPF inválido (dígito verificador)
111.111.111-11	CPF inválido (sequência repetida)
11.222.333/0001-81	CNPJ numérico válido
11.222.333/0001-82	CNPJ inválido (dígito verificador)
12.ABC.345/01DE-35	CNPJ alfanumérico válido (exemplo oficial da Receita)
12abc34501de35	CNPJ alfanumérico válido depois de normalizado

8. API REST e documentação

A API tem 8 rotas de clientes sob `/api/v1`, mais um healthcheck. Erros seguem o padrão Problem Details (RFC 9457) e a documentação interativa fica em `/docs`.

Convenções

- JSON em UTF-8; ids em UUID (id mal formado: 400).
- Cadastro responde 201 com o header `Location` apontando para o novo cliente.
- Erros em `application/problem+json` (RFC 9457), com a lista de campos inválidos quando houver.
- Erro 500 nunca expõe stack trace nem mensagem do Prisma; o detalhe vai para o log.
- Listagens paginadas no servidor: `page` (padrão 1), `pageSize` (padrão 10, máximo 50), `search` (nome, e-mail ou documento, sem diferenciar maiúsculas), `sortBy` e `sortOrder`.
- Listagens devolvem os itens em `data` e a paginação em `meta` (`page`, `pageSize`, `total`, `totalPages`).
- A interface chama a API por caminho relativo (`/api`): proxy do Nginx no container e do Vite em desenvolvimento, sem depender de CORS.

Rotas

Método	Rota	O que faz	Sucesso	Erros
GET	`/health`	Confere a API e a conexão com o banco	200	503
GET	`/api/v1/customers`	Lista clientes ativos	200	400
GET	`/api/v1/customers/deleted`	Lista clientes na lixeira	200	400
GET	`/api/v1/customers/{id}`	Detalha um cliente ativo	200	400, 404
POST	`/api/v1/customers`	Cadastra um cliente	201	400, 409
PATCH	`/api/v1/customers/{id}`	Edita um cliente ativo	200	400, 404

Método	Rota	O que faz	Sucesso	Erros
DELETE	`/api/v1/customers/{id}`	Move para a lixeira (soft delete)	204	404
POST	`/api/v1/customers/{id}/restore`	Restaura da lixeira	200	404, 409
DELETE	`/api/v1/customers/{id}/permanent`	Exclui definitivamente	204	404

Exemplos

Cadastro de PF (`POST /api/v1/customers`). Para PJ, `personType` é COMPANY, o CNPJ vai em `document`, `tradeName` é opcional e não existe `birthDate`.

```
{
  "personType": "INDIVIDUAL",
  "name": "Maria da Silva",
  "document": "529.982.247-25",
  "birthDate": "1990-05-14",
  "email": "Maria.Silva@Exemplo.com.br",
  "phone": "(11) 98765-4321",
  "address": {
    "zipCode": "01310-100",
    "street": "Avenida Paulista",
    "number": "1000",
    "complement": "Apto 12",
    "district": "Bela Vista",
    "city": "São Paulo",
    "state": "SP"
  }
}
```

Resposta 201: o cliente completo, já normalizado.

```
{
  "id": "6f1c2b9e-3d4a-4c8b-9a7e-1f2d3c4b5a69",
  "personType": "INDIVIDUAL",
  "name": "Maria da Silva",
  "document": "52998224725",
  "birthDate": "1990-05-14",
  "email": "maria.silva@exemplo.com.br",
```

```
  "phone": "11987654321",
  "address": {
    "zipCode": "01310100",
    "street": "Avenida Paulista",
    "number": "1000",
    "complement": "Apto 12",
    "district": "Bela Vista",
    "city": "São Paulo",
    "state": "SP"
  },
  "createdAt": "2026-10-05T13:45:00.000Z",
  "updatedAt": "2026-10-05T13:45:00.000Z",
  "deletedAt": null
}
```

Erro de validação (400):

```
{
  "type": "/problems/validation-error",
  "title": "Dados inválidos",
  "status": 400,
  "detail": "Um ou mais campos não passaram na validação.",
  "instance": "/api/v1/customers",
  "errors": [
    { "field": "document", "message": "CPF inválido" },
    { "field": "address.zipCode", "message": "CEP deve ter 8 dígitos" }
  ]
}
```

Documento de um cliente que está na lixeira (409):

```
{
  "type": "/problems/customer-in-trash",
  "title": "Cliente na lixeira",
  "status": 409,
  "detail": "Já existe um cliente com este documento na lixeira.",
  "instance": "/api/v1/customers",
  "customerId": "6f1c2b9e-3d4a-4c8b-9a7e-1f2d3c4b5a69"
}
```

Edição (`PATCH`): campos omitidos ficam como estão e `address` aceita atualização parcial.

```
{
  "phone": "11912345678",
  "address": { "number": "1200" }
}
```

Documentação

- OpenAPI 3 gerado automaticamente a partir dos schemas Zod; especificação escrita à mão não é aceita.
- UI interativa em `http://localhost:3333/docs`, com a especificação em JSON disponível para download.
- Toda rota com resumo, descrição, tag, parâmetros, corpo, todas as respostas possíveis (inclusive erros) e exemplos.
- Aceite: com o sistema no ar via `start.sh`, todos os endpoints podem ser executados pela própria UI de documentação.

9. Interface web

A interface tem 4 telas, funciona do celular ao desktop e trata carregamento, vazio e erro em todas elas. Textos em português, com mensagens de erro junto ao campo.

Telas e rotas

Rota	Tela	Conteúdo
`/`	Clientes	Listagem de ativos com busca, ordenação, paginação e ações
`/clientes/novo`	Novo cliente	Wizard de cadastro em 4 etapas
`/clientes/:id/editar`	Editar cliente	Formulário com os dados atuais
`/lixeira`	Lixeira	Clientes excluídos, com restaurar e excluir definitivamente
Qualquer outra	Não encontrado	Página 404 com link para a listagem

Listagem (página principal)

- Tabela no desktop; abaixo de 768 px (breakpoint `md` do Tailwind), cada cliente vira um card.
- Mostra nome (com nome fantasia abaixo, para PJ), tipo, documento e telefone formatados, e-mail, cidade/UF e data de cadastro.
- Busca com debounce de 300 ms por nome, e-mail ou documento, com ou sem máscara.
- Página, busca e ordenação ficam na URL: recarregar ou compartilhar o link mantém o estado.
- Ações por cliente: Editar, Mover para a lixeira e Excluir definitivamente. Botão "Novo cliente" sempre visível.
- Estados: carregando (skeleton), lista vazia (com atalho para cadastrar), busca sem resultado e erro (com "Tentar novamente").

Wizard de cadastro

1. **Identificação:** Pessoa Física ou Pessoa Jurídica; os campos mudam conforme o tipo (PF: nome completo, CPF e data de nascimento; PJ: razão social, nome fantasia e CNPJ).
2. **Contato:** e-mail e telefone.
3. **Endereço:** CEP, logradouro, número, complemento, bairro, cidade e UF (lista com as 27 UFs).
4. **Revisão:** resumo de tudo, com atalho para editar cada bloco, e o botão "Cadastrar".

Regras do wizard:

- Indicador de progresso ("Etapa 2 de 4").
- Só avança com a etapa válida; erros aparecem junto ao campo, em português.
- Voltar não perde nada. Trocar de PF para PJ limpa só os campos que não valem para o novo tipo.
- Máscaras para CPF, CNPJ (inclusive alfanumérico), telefone e CEP.
- Botão de envio desabilitado durante a requisição, para não cadastrar duas vezes.
- Sucesso: aviso de confirmação (toast) e volta à listagem com o novo cliente visível.
- Erro 409: mensagem no campo documento; se o cliente estiver na lixeira, oferecer "Restaurar cadastro existente".

Edição

- Reaproveita os componentes e as validações do wizard, em página única com seções.
- Tipo de pessoa e documento aparecem bloqueados.
- Salva via `PATCH`; depois, toast e volta à listagem.
- Se o cliente não existir mais (404), avisa e volta à listagem.

Exclusões e lixeira

- **Mover para a lixeira:** diálogo de confirmação simples, depois toast de sucesso.
- **Excluir definitivamente:** diálogo destrutivo que avisa que a ação é irreversível; o botão só habilita depois de digitar EXCLUIR.
- **Lixeira:** clientes excluídos com a data da exclusão, busca e paginação; ações Restaurar e Excluir definitivamente; estado vazio próprio.
- Depois de cada ação, as listas afetadas se atualizam sem recarregar a página.

Celular e acessibilidade

- Mobile first, sem rolagem horizontal da página entre 360 px e 1440 px de largura.
- Validado em 360 × 640, 390 × 844 e 1280 × 800.
- Navegação compacta no celular (menu ou barra inferior).
- Alvos de toque com pelo menos 44 px; no celular, os botões do wizard ficam fixos no rodapé.
- Todo campo com label, foco visível e diálogos operáveis por teclado (foco preso no diálogo, Esc fecha).
- Contraste mínimo WCAG AA.

10. Tipagem forte

Dado inválido não compila ou não passa da fronteira: os schemas Zod são a única fonte dos tipos e o compilador roda no modo mais estrito em todos os pacotes.

- `tsconfig` base compartilhado, com no mínimo estas opções:

```
{
  "compilerOptions": {
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "noImplicitOverride": true,
    "noImplicitReturns": true,
    "noFallthroughCasesInSwitch": true
  }
}
```

- Proibido no código (lint em modo erro): `any` explícito, `@ts-ignore` e non-null assertion (`!`). Um `@ts-expect-error` inevitável leva justificativa.
- Asserções `as` só em fronteiras, com comentário explicando; `as const` é livre.
- Schemas Zod são a fonte única: os tipos saem de `z.input` e `z.output` (ou `z.infer`). Interface escrita à mão que duplica um schema não é aceita.
- Schemas de entrada rejeitam campos desconhecidos (modo estrito do Zod).
- Cliente modelado como união discriminada por `personType`, na API e na interface, com `switch` exaustivo (verificação `never` no `default`).
- Branded types para `CustomerId`, `Cpf` e `Cnpj`: só valores validados circulam com esses tipos.
- Variáveis de ambiente validadas com Zod na inicialização; valor ausente ou inválido derruba a aplicação com mensagem clara.
- Toda rota declara o schema de resposta de cada status code, e o retorno do handler é checado pelo compilador.
- Tipos do Prisma não saem da camada de repositório: mappers convertem a linha do banco em tipo de domínio.
- Na interface: cliente HTTP tipado (nada de `fetch` devolvendo `any`), respostas validadas com os schemas do pacote de contratos e formulários com zodResolver usando os mesmos schemas da API.
- `tsc --noEmit` passa sem erros em todos os pacotes, e o build falha se houver erro de tipo.

Exemplo ilustrativo do que se espera no pacote de contratos (a sintaxe exata depende da versão do Zod):

```
export const cpfSchema = z.string().transform(onlyDigits).refine(isValidCpf,
'CPF inválido').brand<'Cpf'>();

const individualSchema = z.object({
  personType: z.literal('INDIVIDUAL'),
  name: nameSchema,
  document: cpfSchema,
  birthDate: birthDateSchema,
  ...contactAndAddressShape,
});

const companySchema = z.object({
  personType: z.literal('COMPANY'),
  name: nameSchema,
  tradeName: tradeNameSchema.optional(),
  document: cnpjSchema,
  ...contactAndAddressShape,
});

export const createCustomerSchema = z.discriminatedUnion('personType',
[individualSchema, companySchema]);
export type CreateCustomerInput = z.input<typeof createCustomerSchema>; // o
que o formulário envia
export type CreateCustomer = z.output<typeof createCustomerSchema>; // o que
o service recebe

export function displayName(customer: Customer): string {
  switch (customer.personType) {
    case 'INDIVIDUAL':
      return customer.name;
    case 'COMPANY':
      return customer.tradeName ?? customer.name;
    default:
      return assertNever(customer);
  }
}
```

11. Banco de dados e migrations

O banco é o PostgreSQL 18 na imagem oficial alpine (a variante leve), com os dados em um volume nomeado. Nada se perde ao reiniciar, recriar containers ou rodar o `start.sh` de novo.

Persistência

- Imagem `postgres:18-alpine`, com a tag de versão fixa no `docker-compose.yml`.
- Volume nomeado (ex.: `pgdata`) montado em `/var/lib/postgresql`. A partir da versão 18 esse é o caminho certo; tutoriais antigos montam em `/var/lib/postgresql/data`, e aí os dados vão para um volume anônimo que some quando o container é recriado (documentação da imagem oficial).
- Dados só são apagados por comando explícito (`./start.sh --reset` ou `docker compose down -v`).
- Credenciais em variáveis de ambiente: `.env.example` versionado, `.env` no `.gitignore`, nenhum segredo no código.
- Healthcheck com `pg_isready`; a API só sobe depois do banco saudável.

Migrations

- Prisma Migrate, com as migrations versionadas em `apps/api/prisma/migrations`.
- Em desenvolvimento, `prisma migrate dev`; no container, a API roda `prisma migrate deploy` antes de iniciar o servidor.
- `prisma db push` não entra no fluxo do projeto.
- Migration aplicada não é editada: cada mudança de schema gera uma nova. O histórico precisa mostrar a evolução real do modelo (não vale apagar tudo e gerar uma migration única no fim).
- Busca por nome e e-mail ignora maiúsculas e minúsculas; busca por documento compara o valor normalizado.

Aceite: cadastrar um cliente, rodar `docker compose down`, rodar `./start.sh` de novo e o cliente continuar lá.

12. Containerização e start.sh

Um clone limpo do repositório, numa máquina só com Docker, sobe tudo com `./start.sh`. Sem passos manuais e sem instalar Node na máquina.

Containers

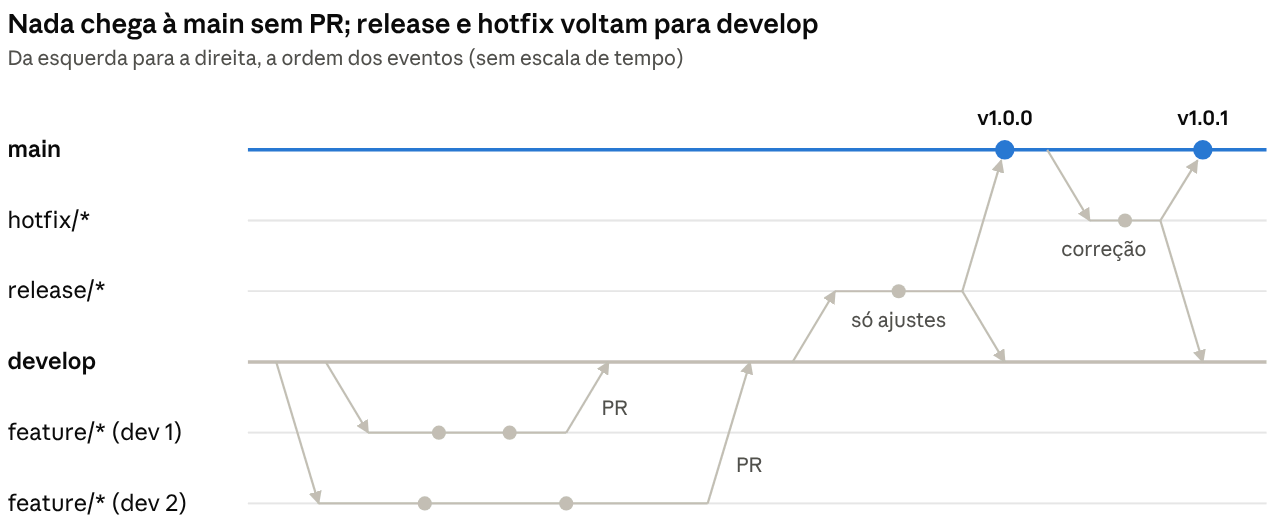
- `docker-compose.yml` com três serviços: `db`, `api` e `web`.
- Dockerfiles multi-stage para `api` e `web`; imagens finais sem dependências de desenvolvimento.
- A imagem da API roda com usuário não root e aplica as migrations antes de subir o servidor.
- `web`: build do Vite servido pelo Nginx, com fallback de SPA (recarregar `/lixeira` não dá 404) e proxy de `/api` para o serviço `api`.
- Ordem de subida garantida por `depends_on` com `condition: service_healthy`; a API expõe `GET /health` para o healthcheck.
- Portas configuráveis no `.env`, com padrão: interface em `http://localhost:8080` e API em `http://localhost:3333`. O banco não precisa ficar exposto ao host.
- No dia a dia, API e interface podem rodar fora do container com hot reload, usando só o banco via Compose. O README explica os dois modos.

start.sh

- Começa com `#!/usr/bin/env bash` e `set -euo pipefail`.
- Confere os pré-requisitos (Docker instalado, daemon rodando, Docker Compose v2) e falha com mensagem clara se faltar algo.
- Cria o `.env` a partir do `.env.example` quando ele não existe.
- Faz build e sobe os serviços em segundo plano (`docker compose up -d --build`).
- Espera a API ficar saudável, com tempo limite. Se estourar, mostra os logs relevantes e sai com código diferente de zero.
- Ao final, imprime os endereços da interface, da API e da documentação.
- É idempotente: rodar duas vezes seguidas não quebra nada nem apaga dados.
- Funciona em Linux, macOS e Windows com WSL2.
- Opções como `--help`, `--stop`, `--logs`, `--seed` e `--reset` (com confirmação) contam como diferencial.

13. Fluxo de trabalho com Gitflow

Todo código chega à `main` por Pull Request revisado pelo outro dev. Features saem de `develop`; releases e hotfixes chegam à `main` com tag de versão e voltam para `develop`.



Gitflow · features em paralelo, release 1.0.0 e hotfix 1.0.1

Cada dev trabalha na própria feature, e o outro revisa o PR. A `main` só recebe código por release ou hotfix, sempre com tag, e tudo volta para `develop`.

Regras

- Commit direto em `main` e `develop` é bloqueado por branch protection no GitHub.
- Nomes de branch: `feature/<issue>-<descricao>` (ex.: `feature/12-customer-wizard`), `release/<versão>` e `hotfix/<versão>`.
- Toda mudança entra por Pull Request, com aprovação obrigatória do outro dev antes do merge.
- PRs pequenos e focados: um por issue, idealmente até cerca de 400 linhas alteradas (sem contar arquivos gerados).
- Template de PR com o que muda, como testar, prints ou vídeo para mudanças de interface e checklist (lint, typecheck, build).
- Revisão de verdade: comentários com sugestões e pedidos de ajuste, não só "aprovado".
- Commits no padrão Conventional Commits (`feat:`, `fix:`, `refactor:`, `docs:`, `test:`, `chore:`, `ci:`, `build:`).
- Release: `release/x.y.z` sai de `develop` só com ajustes finais e CHANGELOG; entra na `main` com tag anotada (`v1.0.0`) e volta para `develop`.

Planejamento

- Backlog no GitHub Projects com as colunas Backlog, A fazer, Em andamento, Em revisão e Concluído.
- Cada requisito vira uma ou mais issues com critério de aceite; o PR fecha a issue (`Closes #12`).
- Os dois devs contribuem nas duas camadas: cada um precisa de ao menos um PR relevante na API e um na interface.

Hotfix obrigatório

Depois da tag v1.0.0, os avaliadores abrem uma issue de bug. A dupla corrige pelo fluxo de hotfix (`hotfix/1.0.1` a partir da `main`, merge em `main` e `develop`, tag `v1.0.1`) em até 1 dia útil.

14. Clean code

Cada camada tem uma responsabilidade só, e qualquer pessoa do time acha onde mora uma regra sem precisar perguntar.

API em camadas

- **routes:** declara rotas, schemas e status codes; não tem regra de negócio nem acesso ao banco.
- **services:** regras de negócio (unicidade, imutabilidade, lixeira, exclusão).
- **repositories:** único lugar que usa o Prisma; devolve tipos de domínio via mappers.
- **errors:** erros de domínio tipados (ex.: `CustomerNotFoundError`, `DocumentConflictError`), convertidos em Problem Details por um error handler central.
- Configuração centralizada (env validado) e logger estruturado (pino, já integrado ao Fastify), sem `console.log`.

Estrutura sugerida:

```
apps/api/src/
  config/env.ts
  modules/customers/
    customer.routes.ts
    customer.service.ts
    customer.repository.ts
    customer.mapper.ts
    customer.errors.ts
  shared/http/error-handler.ts
  app.ts
  server.ts
```

Interface

- Organização por funcionalidade (`features/customers` com `components` , `hooks` , `api` e `pages`); componentes genéricos em `components/ui` .
- Acesso a dados em hooks (TanStack Query), não espalhado pelas telas.
- Componentes pequenos, reaproveitados entre wizard e edição.

Geral

- Nomes descritivos e consistentes; funções curtas, com uma responsabilidade.
- Sem números e textos mágicos: constantes nomeadas (ex.: `MAX_PAGE_SIZE = 50`).
- Sem código morto, comentado ou `console.log` esquecido.
- Lint e formatação sem erros nem avisos em todos os pacotes.
- README com: como rodar, arquitetura, decisões tomadas (e por quê) e o que ficou de fora.

15. Diferenciais

Itens opcionais que somam até 10 pontos extras (seção 16). Só contam se todos os requisitos obrigatórios estiverem atendidos.

- Testes unitários dos validadores e services, e de integração das rotas com banco de teste (Vitest).
- Testes end-to-end do fluxo principal (Playwright).
- CI no GitHub Actions rodando lint, typecheck, testes e build em todo PR, como check obrigatório para o merge.

- Endereço preenchido automaticamente pelo CEP (ViaCEP), com digitação manual se a consulta falhar.
- Toast com "Desfazer" depois de mover para a lixeira.
- Confirmação ao sair do wizard com dados não salvos.
- Controle de concorrência na edição (ETag com `If-Match`, ou `updatedAt`), com 409 ou 412 em caso de conflito.
- Constraint CHECK no banco garantindo a coerência entre tipo de pessoa e campos (SQL na migration).
- `exactOptionalPropertyTypes` habilitado no tsconfig.
- Spec OpenAPI exportada para o repositório e conferida no CI.
- Seed com clientes de exemplo (`./start.sh --seed`) e as demais opções do script.
- ShellCheck sem avisos no `start.sh`.
- Tema escuro.

16. Entrega e avaliação

A nota vai de 0 a 100, mais até 10 pontos de diferenciais, e só é calculada se o sistema subir com `./start.sh` numa máquina limpa. Sem isso, a avaliação técnica para nesse ponto.

Entregáveis

- Repositório no GitHub com acesso para os avaliadores.
- Tags `v1.0.0` e `v1.0.1` na `main`, com CHANGELOG.
- README completo (seção 14).
- Quadro do GitHub Projects com o histórico das issues.
- Apresentação de 30 minutos: demo no desktop e no celular, passeio pelos PRs e decisões, e perguntas. Cada dev explica uma parte que fez e uma parte que o outro fez.

Pesos

Critério	Peso (pontos)	O que é observado
Funcionalidades	20	Requisitos das seções 7 e 9 funcionando, sem bugs nos fluxos principais

Critério	Peso (pontos)	O que é observado
Fluxo de trabalho	15	Gitflow, PRs pequenos, revisões reais, commits, issues, releases e hotfix
Tipagem forte	15	Seção 10 atendida, contratos compartilhados, zero `any`
Clean code	15	Camadas, nomes, coesão, tratamento de erros, lint limpo
API REST e documentação	10	Rotas, status codes, Problem Details, OpenAPI completo e executável
Banco e migrations	10	Modelagem, constraints, histórico de migrations, persistência
Interface e responsividade	10	Fluxos completos, estados de tela, celular, acessibilidade
Containerização e start.sh	5	Dockerfiles, Compose, healthchecks e robustez do script

Escala

Cada critério recebe nota de 0 a 4, convertida em pontos pelo peso (nota ÷ 4 × peso):

- **0:** ausente.
- **1:** insuficiente, com falhas graves.
- **2:** parcial.
- **3:** atende ao documento.
- **4:** vai além, com qualidade.

Métricas do fluxo de trabalho

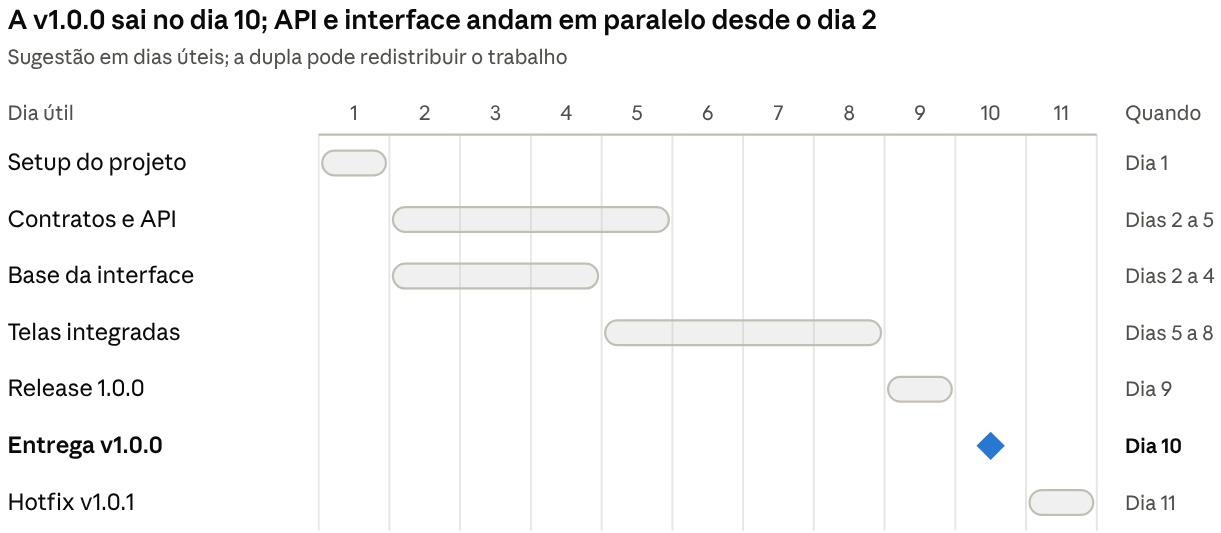
Coletadas no GitHub ao final, para apoiar a nota de fluxo de trabalho e a conversa da apresentação:

- Quantidade de PRs e tamanho médio (linhas alteradas).
- Tempo até a primeira revisão e até o merge.
- Comentários de revisão por PR, e quantos geraram mudança.

- Percentual de commits no padrão Conventional Commits.
- Issues fechadas por PR vinculado.
- Divisão das contribuições entre os dois devs, na API e na interface.
- Tempo do hotfix, da abertura da issue até a tag `v1.0.1`.

17. Cronograma sugerido

São 10 dias úteis até a v1.0.0, com API e interface em paralelo a partir do dia 2, e o hotfix no dia útil seguinte à entrega.



cronograma · 10 dias úteis até a v1.0.0, mais 1 de hotfix

A interface começa pela base (layout, componentes e cliente HTTP) enquanto a API fica pronta; as telas se integram à API a partir do dia 5.

18. Checklist de aceite final

A entrega está pronta quando todos os itens abaixo estiverem marcados.

- [ ] `git clone` seguido de `./start.sh` sobe tudo, sem passos manuais
- [ ] Listagem, wizard, edição, lixeira, soft delete, restauração e exclusão definitiva funcionando
- [ ] Interface utilizável com 360 px de largura
- [ ] Dados continuam lá depois de `docker compose down` e um novo `./start.sh`
- [ ] Migrations versionadas e aplicadas automaticamente na subida

- [ ] Todos os endpoints executáveis em `/docs`
- [ ] Typecheck e lint sem erros em todos os pacotes
- [ ] Nenhum `any` no código
- [ ] Casos de teste dos validadores (seção 7) passando
- [ ] Tags `v1.0.0` e `v1.0.1` na `main`, com CHANGELOG
- [ ] Todo merge em `develop` e `main` feito por PR aprovado
- [ ] README com instruções, arquitetura e decisões

19. Resultado da parte 1

A v1 foi entregue e avaliada pelos pesos da seção 16. As notas abaixo fecham a parte 1; os pontos de atenção viram requisitos obrigatórios da parte 2 (seção 21).

Critério	Peso	Pontos obtidos
Funcionalidades	20	13
Fluxo de trabalho	15	7
Tipagem forte	15	20
Clean code	15	20
API REST e documentação	10	7
Banco e migrations	10	10
Interface e responsividade	10	10
Containerização e start.sh	5	5
Total	**100**	**92**

Pontos de atenção

1. O cliente não pode ver erros no console do navegador ao acessar o sistema.
2. A versão entregue deve ser a compilada (build de produção), não a que roda em modo dev.
3. O complemento do endereço deve ser opcional. O número pode continuar como estava.

Os critérios com nota mais baixa (fluxo de trabalho, funcionalidades e API) são o foco natural da parte 2.

20. Parte 2: objetivo e formato

A mesma dupla evolui o mesmo repositório a partir da v1.0.1: o sistema ganha autenticação com JWT, multi tenant e permissões, logs centralizados, testes automatizados na API e na interface, seed de dados e as correções apontadas na avaliação da parte 1.

- **Prazo sugerido:** 13 dias úteis até a release v2.0.0. Ajustável.
- **Versão:** v2.0.0, porque as rotas de clientes passam a exigir login, o que quebra quem consumia a API sem autenticação (versionamento semântico: mudança incompatível sobe o major).
- **Mudança de escopo:** login, perfis, permissões e multi tenant saem da lista "Fora do escopo" da seção 3. Cada empresa (tenant) vê só os próprios dados, e cada usuário tem um papel que define o que pode fazer (seção 22).
- **O que continua valendo:** todas as regras da parte 1 (stack, tipagem, clean code, Gitflow, idioma, start.sh, Problem Details e OpenAPI). Código novo segue o mesmo padrão; regressão em algo que já funcionava conta contra.

#	Requisito obrigatório	Detalhado em
P1	Correções da parte 1	Seção 21
P2	Autenticação com JWT, rotas públicas e autenticadas, multi tenant e permissões, na API e na interface	Seção 22
P3	Logs de todos os containers centralizados por um agente dedicado	Seção 23
P4	Testes unitários de todos os endpoints	Seção 24
P5	Sistema de seed	Seção 25
P6	Automação de QA da interface	Seção 26
P7	Problemas debatidos na reunião	Seção 27

21. Correções obrigatórias da parte 1

Os três pontos de atenção da avaliação são pré-requisito da parte 2: sem eles, o restante não é avaliado.

21.1 Console limpo

- Abrir qualquer tela, logado ou não, na versão compilada, não gera nenhum erro nem aviso no console do navegador: nada de exceção sem tratamento, promessa rejeitada sem `catch`, aviso do React, asset ou favicon com 404, nem requisição com falha esperada.
- Erro de requisição aparece para o usuário como estado de erro ou toast, nunca só no console.
- Error boundary global com tela amigável e botão para recarregar.
- `console.log`, `console.warn` e `console.error` proibidos no código da interface (regra de lint `no-console` em modo erro).
- Cuidado com a autenticação: verificar se existe sessão não pode responder 401 no primeiro acesso, porque o navegador registra a falha no console. Por isso a rota de sessão (seção 22) responde 200 com usuário nulo.
- **Aceite:** a automação de QA (seção 26) reprova qualquer teste que encontre erro ou aviso no console.

21.2 Entrega compilada

- `web`: o container serve só os arquivos do `vite build` pelo Nginx. Servidor de desenvolvimento do Vite não entra na imagem final.
- `api`: o TypeScript é compilado no estágio de build e o container roda `node dist/server.js` com `NODE_ENV=production`. Nada de `tsx`, `ts-node`, `nodemon` ou modo watch na imagem final.
- Imagens finais sem dependências de desenvolvimento e sem o código-fonte TypeScript.
- O modo de desenvolvimento com hot reload continua existindo, mas só fora do `start.sh` (seção 12).
- **Aceite:** com o sistema no ar via `start.sh`, o processo principal do `web` é o Nginx, o da `api` é `node dist/...`, e o navegador não carrega nenhum arquivo do Vite de desenvolvimento (como `/@vite/client`).

21.3 Endereço: complemento opcional

- `complement` é opcional no schema Zod, no banco, no wizard e na edição, como define a seção 6. Campo vazio é salvo como nulo, não como texto vazio.
- A interface não marca o campo como obrigatório, e a revisão do wizard mostra "Não informado" quando ele está vazio.
- `number` continua como estava: obrigatório, até 10 caracteres, aceitando "S/N".
- Se a coluna foi criada como obrigatória, a correção entra em uma migration nova (migration aplicada não é editada, seção 11).
- **Aceite:** cadastrar e editar um cliente sem complemento funciona pela interface e pela documentação da API.

22. Autenticação, multi tenant e permissões

O sistema passa a ter usuários com e-mail e senha, cada um ligado a uma empresa (tenant) e a um papel. Login, cadastro de conta, documentação e healthcheck são públicos; todo o resto, na API e na interface, exige sessão válida e a permissão certa. Esta seção vem em três partes: autenticação, JWT e multi tenant com permissões.

Modelo de dados

Tabelas novas de autenticação, criadas por migrations novas (a tabela de tenants e as mudanças em clientes estão mais abaixo, em Multi tenant):

Tabela	Campos	Regras
`users`	id (UUID), tenantId, role, name, email, passwordHash, createdAt, updatedAt	name de 3 a 100 caracteres; email único em toda a base, salvo em minúsculas; role é ADMIN, OPERATOR ou VIEWER; a senha nunca é salva em texto
`sessions`	id (UUID), userId, refreshTokenHash, expiresAt, revokedAt, createdAt	Uma linha por login; o refresh token é salvo só como hash; sessão revogada ou vencida não renova

Regras de segurança

- Senha com 8 a 128 caracteres, guardada com hash argon2id. Senha e hash nunca aparecem em resposta, log ou documentação.
- Access token JWT curto (15 minutos) e refresh token opaco (7 dias), os dois em cookies `httpOnly`, `SameSite=Strict` e `Secure` fora do ambiente local. Token nunca vai para `localStorage` nem para a URL.
- Cada uso do refresh token gera um novo e invalida o anterior (rotação).
- Segredo do JWT no `.env`, validado na inicialização com Zod (mínimo de 32 caracteres).
- Login com limite de tentativas: 5 por minuto por IP e e-mail, com 429 ao estourar.
- Erro de login genérico ("E-mail ou senha inválidos"), sem revelar se o e-mail existe.

Rotas novas da API

Método	Rota	Acesso	O que faz	Sucesso	Erros
POST	`/api/v1/auth/register`	Pública	Cria um tenant novo, o usuário como ADMIN dele e já inicia a sessão	201	400, 409
POST	`/api/v1/auth/login`	Pública	Valida e-mail e senha e grava os cookies	200	400, 401, 429
POST	`/api/v1/auth/refresh`	Pública, com cookie de refresh	Renova a sessão	200	401
POST	`/api/v1/auth/logout`	Pública e idempotente	Revoga a sessão e limpa os cookies	204	Nenhum
GET	`/api/v1/auth/session`	Pública	Devolve o usuário logado, com tenant, papel e permissões, ou `null`	200	Nenhum

- `/health` e `/docs` continuam públicos.
- As 8 rotas de clientes passam a exigir sessão. Sem sessão válida: 401 em Problem Details, com `type` `/problems/unauthenticated`.
- A proteção é aplicada uma vez, no escopo do módulo protegido (hook do Fastify em plugin encapsulado), e não repetida rota a rota.
- `request.user` é tipado por declaration merging do Fastify, sem `any` e sem asserção.
- Schemas de autenticação ficam no pacote de contratos, como os de clientes.

- OpenAPI com o esquema de segurança por cookie, rotas protegidas marcadas e a resposta 401 documentada. **Aceite:** em `/docs`, fazer login pela própria documentação e depois executar todas as rotas de clientes.

Interface

Rota	Tela	Acesso
`/entrar`	Login	Pública; quem já está logado vai para `/`
`/criar-conta`	Cadastro de conta e da empresa	Pública; quem já está logado vai para `/`
`/`, `/lixeira`	Listagem e lixeira	Autenticadas, permissão `customer:read`
`/clientes/novo`, `/clientes/:id/editar`	Wizard e edição	Autenticadas, permissão `customer:write`
`/usuarios`	Usuários do tenant	Autenticada, permissão `user:manage`
Qualquer outra	Não encontrado	Pública

- Ao abrir o sistema, a interface consulta `/auth/session` e mostra um estado de carregamento até saber se há sessão. Conteúdo protegido nunca pisca na tela antes do redirecionamento.
- Acesso a rota protegida sem sessão leva para `/entrar?redirect=<rota>`; depois do login, o usuário volta para onde estava. O `redirect` só aceita caminhos internos (nada de URL externa).
- O cliente HTTP trata 401: tenta renovar a sessão uma vez (requisições simultâneas compartilham a mesma renovação) e repete a chamada. Se a renovação falhar, limpa o cache do TanStack Query e leva para o login com o aviso "Sua sessão expirou".
- Cabeçalho com o nome do usuário e o botão "Sair"; sair limpa o cache e volta para `/entrar`.
- Formulários de login e de conta com zodResolver e os schemas do pacote de contratos, mensagens em português junto ao campo, botão de mostrar senha e atributos `autocomplete` corretos.
- As telas novas seguem as regras de celular e acessibilidade da seção 9.

JWT: o que é e como implementar

JWT (JSON Web Token) é um texto em três partes separadas por ponto: cabeçalho, conteúdo (as claims) e assinatura. A API assina o token com um segredo que só ela conhece; quando o token volta, ela confere a assinatura e sabe que ninguém alterou o conteúdo, sem consultar o banco. Atenção: assinado não é criptografado. Qualquer pessoa consegue ler o conteúdo de um JWT, então nada sensível vai nele.

Neste sistema o JWT é só o access token, de vida curta. A renovação usa um refresh token opaco (texto aleatório guardado como hash na tabela `sessions`), porque ele precisa poder ser revogado no logout, e um JWT não pode ser "desligado" antes de vencer.

Item	Definição
Biblioteca	`@fastify/jwt`, lendo o token do cookie com `@fastify/cookie`
Algoritmo	HS256, com o segredo `JWT_SECRET` do `.env` (mínimo de 32 caracteres, validado com Zod na inicialização)
Claims	`sub` (id do usuário), `tid` (id do tenant), `role` (papel), `sid` (id da sessão), `iat` e `exp`
Validade	15 minutos, com tolerância de 5 segundos para diferença de relógio
Transporte	Cookie `access_token`, `httpOnly`, `SameSite=Strict`, `Secure` fora do ambiente local, `Path=/api`
Proibido no payload	Senha, hash, e-mail, documento ou qualquer dado de cliente

Regras de implementação:

- Depois de verificar a assinatura, o payload passa por um schema Zod do pacote de contratos (`accessTokenClaimsSchema`) antes de virar `request.user`. Token com assinatura válida mas formato inesperado é tratado como inválido.
- Gerar e verificar token fica num único módulo (ex.: `modules/auth/token.service.ts`); rotas e services não chamam a biblioteca de JWT diretamente.
- Token vencido, adulterado ou ausente responde 401 com o mesmo Problem Details `unauthenticated`, sem dizer qual foi o motivo ao cliente (o motivo vai para o log).

- No login e no refresh, a API emite um access token novo e um refresh token novo; o refresh antigo é marcado como usado.
- No logout, a sessão (`sid`) é revogada. O access token ainda vale até vencer, por no máximo 15 minutos; essa limitação fica registrada no README.
- Diferencial: RS256 com par de chaves, publicando a chave pública em `/.well-known/jwks.json` .

Fluxo completo:

1. Login correto: a API cria a linha em `sessions` , assina o JWT e grava os dois cookies.
2. Cada requisição: o navegador envia o cookie sozinho; a API verifica o JWT e preenche `request.user` .
3. Depois de 15 minutos: a API responde 401, a interface chama `/auth/refresh` , recebe tokens novos e repete a chamada original.
4. Logout ou refresh vencido: a sessão acaba e a interface volta para o login.

Multi tenant: o que é e como funciona

Tenant é cada empresa que usa o sistema. Todas usam a mesma aplicação e o mesmo banco, mas cada uma só enxerga os próprios dados, como os apartamentos de um prédio: estrutura comum, chave diferente para cada porta. Aqui, todo usuário e todo cliente cadastrado pertencem a um tenant, e o tenant de quem faz a requisição vem sempre do JWT (claim `tid`), nunca de algo que o navegador mande.

Há três jeitos comuns de separar os dados: um banco por tenant, um schema por tenant ou uma coluna `tenant_id` nas tabelas compartilhadas. Usamos a coluna: é a mais simples de operar e a mais comum em sistemas desse porte. O preço é que toda consulta precisa filtrar pelo tenant, e esquecer o filtro mostra dados de outra empresa. Por isso o filtro mora num lugar só (o repositório) e tem teste dedicado.

Permissionamento responde outra pergunta: dentro do próprio tenant, o que este usuário pode fazer? Usamos papéis (RBAC): cada usuário tem um papel, e cada papel libera uma lista de permissões. O código pergunta pela permissão ("pode excluir definitivamente?"), não pelo nome do papel. Assim, mudar o que um papel faz é editar uma tabela, não caçar `if` pelo código.

Caminho de uma requisição protegida:

1. **Quem é:** o JWT diz o usuário, o tenant e o papel. Sem token válido: 401.

2. **Pode fazer isso:** a rota declara a permissão que exige, e o papel precisa tê-la. Sem permissão: 403.
3. **Em quais dados:** o service chama o repositório passando o `tenantId`, e o repositório filtra por ele. Cliente de outro tenant simplesmente não é encontrado: 404.

Tenants sugeridos

Tenant	Slug	Usuários do seed
Empresa Alfa	`alfa`	`admin@alfa.local` (ADMIN), `operador@alfa.local` (OPERATOR), `leitor@alfa.local` (VIEWER)
Empresa Beta	`beta`	`admin@beta.local` (ADMIN), `operador@beta.local` (OPERATOR), `leitor@beta.local` (VIEWER)

Cada tenant tem os próprios clientes, e um mesmo CPF fica cadastrado nos dois, para provar que a unicidade do documento é por tenant.

Modelo de dados do multi tenant

- Tabela nova `tenants`: id (UUID), name (3 a 100 caracteres), slug (único), createdAt e updatedAt.
- `users.tenant_id` e `customers.tenant_id` obrigatórios, com chave estrangeira e índice. `addresses` segue o cliente.
- O índice único de `customers.document` passa a ser único em `(tenant_id, document)`. A regra RN01 continua valendo, agora dentro do tenant.
- Os clientes que já existem na v1 vão para um tenant padrão numa migration nova, antes de a coluna virar obrigatória. É uma migration de dados: cria a coluna, preenche e só então trava.

Regras de isolamento

- O `tenantId` vem só do JWT. Nenhuma rota aceita tenant no corpo, na URL ou em header; os schemas estritos rejeitam o campo.
- Toda função do repositório de clientes e de usuários recebe o `tenantId` como parâmetro obrigatório, com o branded type `TenantId`. Não existe consulta a essas tabelas sem ele.

- Registro de outro tenant responde 404, igual a inexistente. Responder 403 entregaria que ele existe.
- Listagem, busca, lixeira, total da paginação e conflito de documento consideram só o tenant da sessão.
- Diferencial: Row Level Security do PostgreSQL como segunda barreira.

Papéis e permissões

Permissão	O que libera	ADMIN	OPERATOR	VIEWER
`customer:read`	Listar, detalhar e ver a lixeira	Sim	Sim	Sim
`customer:write`	Cadastrar e editar	Sim	Sim	Não
`customer:trash`	Mover para a lixeira e restaurar	Sim	Sim	Não
`customer:delete-permanent`	Excluir definitivamente	Sim	Não	Não
`user:manage`	Gerenciar usuários do tenant	Sim	Não	Não

- O mapa de papel para permissões fica no pacote de contratos, como constante tipada (`as const`), usada pela API e pela interface.
- Cada rota declara a permissão que exige, e um hook confere antes do handler. Sem permissão: 403 em Problem Details, com `type` `/problems/forbidden`.
- A API é a fonte da verdade; a interface só esconde o que o usuário não pode fazer.
- O papel vai no JWT, então uma troca de papel vale na próxima renovação, em até 15 minutos. Isso fica registrado no README.

Gestão de usuários (só ADMIN)

Método	Rota	O que faz	Sucesso	Erros
GET	`/api/v1/users`	Lista os usuários do tenant	200	401, 403
POST	`/api/v1/users`	Cria usuário no tenant, com papel e senha inicial	201	400, 401, 403, 409

Método	Rota	O que faz	Sucesso	Erros
PATCH	`/api/v1/users/{id}`	Altera nome ou papel	200	400, 401, 403, 404, 409

- Todo tenant tem ao menos um ADMIN: rebaixar o último responde 409 `last-admin`.
- Usuário de outro tenant responde 404.

Permissões na interface

- Botões e itens de menu sem permissão não aparecem: o leitor não vê "Novo cliente", "Editar" nem as ações da lixeira; o operador não vê "Excluir definitivamente".
- Abrir pela URL uma tela sem permissão mostra a página "Sem permissão", com link para a listagem.
- O cabeçalho mostra o nome do tenant e o papel do usuário.
- Um 403 inesperado da API vira toast, sem quebrar a tela.
- Tela `/usuarios` com a lista, o cadastro de usuário e a troca de papel.

Aceite de multi tenant e permissões

- Logado na Alfa, abrir a URL de edição de um cliente da Beta mostra "não encontrado", e a API responde 404.
- O leitor não consegue cadastrar nem pela interface nem por `/docs` (403).
- O mesmo CPF existe na Alfa e na Beta, cada um no seu tenant.

23. Logs centralizados

Os logs de todos os containers vão para um sistema separado, consultável numa tela só, coletados por um agente dedicado. As aplicações só escrevem na saída padrão; quem lê e envia é o agente.

Stack recomendada

Papel	Ferramenta
Agente coletor	Grafana Alloy, lendo os logs dos containers pelo socket do Docker (montado somente leitura)

Papel	Ferramenta
Armazenamento	Grafana Loki, com volume nomeado e retenção configurada (ex.: 7 dias)
Visualização	Grafana, com fonte de dados e dashboard provisionados por arquivo

Promtail não é aceito: chegou ao fim da vida em 2 de março de 2026 e não recebe mais atualizações (aviso oficial da Grafana). Alternativa aceita: Fluent Bit ou Vector como agente, desde que entregue as mesmas garantias.

Requisitos

- Três serviços novos no `docker-compose.yml` (`loki`, `alloy`, `grafana`), com tags de versão fixas e healthchecks. O `start.sh` continua subindo tudo com um comando e imprime o endereço do Grafana no final.
- Um único agente coleta os logs de `db`, `api` e `web`. Nenhuma aplicação envia log direto ao Loki.
- Labels de baixa cardinalidade: `service`, `container` e `level`. Identificadores como `reqId`, `userId` ou `tenantId` ficam no corpo do log, nunca como label.
- API com logs JSON do pino contendo `level`, `time`, `msg`, `reqId`, método, rota, status e tempo de resposta, mais `userId` e `tenantId` quando houver sessão.
- Redação obrigatória no logger: `authorization`, `cookie`, `set-cookie`, `password`, `passwordHash` e qualquer token, inclusive o JWT. **Aceite:** buscar a senha usada num login no Grafana não encontra nada.
- A API devolve o header `x-request-id` em toda resposta, e o Problem Details de erro 500 traz o `requestId`. Buscar esse id no Grafana leva ao log com o detalhe do erro.
- Nginx com log de acesso em JSON.
- Grafana sobe já configurado, sem cliques manuais: fonte Loki provisionada e um dashboard "Visão geral" com logs por serviço, volume por nível, lista de erros 5xx e filtro por tenant. Usuário e senha do Grafana vêm do `.env`.
- Logs sobrevivem a `docker compose down` e a um novo `./start.sh`.
- **Aceite:** provocar um erro na API e encontrá-lo no Grafana, filtrando por serviço, em até 30 segundos.

24. Testes unitários de todos os endpoints

Todo endpoint da API tem teste unitário, rodando sem banco e sem rede, com Vitest.

- **Abrangência:** os 17 endpoints (`/health` , as 8 rotas de clientes, as 5 de autenticação e as 3 de usuários).
- **Forma:** a rota é exercitada com `app.inject()` do Fastify, com o service substituído por dublê de teste. Assim o teste cobre schema de entrada, status code, headers, schema de resposta e conversão de erro de domínio em Problem Details.
- **Casos por endpoint:** o caminho de sucesso e cada status de erro documentado na OpenAPI (400, 401, 403, 404, 409, 429 e 503, conforme a rota).
- **Autenticação e permissão:** toda rota protegida tem um caso sem token, um com token vencido e um com token adulterado (todos 401), e um caso para cada papel sem a permissão exigida (403).
- **JWT:** o módulo de tokens tem testes próprios para emissão, claims, validade e rejeição de payload fora do schema.
- **Resposta conferida pelo contrato:** o corpo de cada resposta é validado com o schema do pacote de contratos, não só comparado com um objeto fixo.
- **Services:** testados à parte, com repositório falso em memória, cobrindo as regras RN01 a RN11, as regras de sessão e a regra do último ADMIN.
- **Isolamento entre tenants:** exceção obrigatória à regra de "sem banco". Um teste de integração com PostgreSQL real prova que o repositório nunca devolve, altera ou conta registro de outro tenant, inclusive na busca e na lixeira.
- **Validadores:** os casos da seção 7 continuam obrigatórios.
- **Cobertura mínima:** 80% de linhas e de branches em `apps/api/src` , configurada no Vitest para falhar abaixo disso.
- **Qualidade dos testes:** independentes entre si e da ordem, datas controladas com fake timers, nomes que descrevem o comportamento ("responde 409 quando o documento está na lixeira").
- `npm test` na raiz roda os testes de todos os pacotes. Os demais testes de integração com banco real continuam valendo como diferencial (seção 15).

25. Sistema de seed

Um comando popula o sistema com dados realistas para testar à mão e para a demonstração.

- Disparo explícito por `./start.sh --seed` (e por um script `npm` equivalente), usando o mecanismo de seed do Prisma. O seed nunca roda sozinho na subida normal.
- **Tenants e usuários:** os dois tenants sugeridos na seção 22 (Empresa Alfa e Empresa Beta), cada um com um usuário por papel. A senha de todos vem do `.env` (`SEED_USER_PASSWORD`), e os e-mails são impressos ao final do comando.
- **Clientes:** cerca de 50 por tenant, cobrindo PF e PJ, CNPJ numérico e alfanumérico, PJ com e sem nome fantasia, endereço com e sem complemento, número "S/N" e cerca de 10 clientes na lixeira. Datas de cadastro espalhadas, para exercitar ordenação e mais de 5 páginas na listagem.
- Um mesmo CPF cadastrado nos dois tenants, para demonstrar a unicidade por tenant.
- Documentos gerados válidos, calculados com os validadores do pacote de contratos.
- **Determinístico:** semente fixa no gerador de dados falsos, então rodar em máquinas diferentes produz os mesmos dados.
- **Idempotente:** rodar duas vezes seguidas não duplica nem quebra (upsert por slug, e-mail e tenant mais documento).
- `./start.sh --reset --seed` volta o sistema ao estado do seed, com a confirmação que o `--reset` já exige.
- A automação de QA (seção 26) usa um conjunto de dados próprio, preparado pelo teste, e não depende do seed manual nem de dados digitados à mão.

26. Automação de QA da interface

Os fluxos da interface são verificados por testes end-to-end com Playwright, rodando contra a mesma versão compilada que é entregue.

Execução

- Pacote próprio no monorepo (ex.: `apps/e2e`), em TypeScript, com as mesmas regras de lint e tipagem.
- Um comando (`npm run test:e2e` ou `./start.sh --qa`) sobe o ambiente com banco isolado, prepara os dados, roda os testes e gera o relatório HTML do Playwright.
- Três perfis: desktop 1280 × 800 e celular 360 × 640 e 390 × 844.
- Em caso de falha, guarda trace, screenshot e vídeo.

Cenários mínimos

1. Acesso a rota protegida sem login leva ao `/entrar` e, depois do login, volta à rota original.
2. Criar conta (com a empresa), errar a senha (mensagem genérica), entrar com a senha certa e sair.
3. Sessão expirada leva ao login com o aviso de sessão expirada; access token vencido com refresh válido renova sem o usuário perceber.
4. Wizard completo de PF e de PJ com CNPJ alfanumérico, inclusive sem complemento.
5. Validações do wizard: CPF e CNPJ inválidos, campos obrigatórios, troca de PF para PJ.
6. Documento já cadastrado e documento na lixeira, com a oferta de restaurar.
7. Busca, ordenação e paginação mantidas na URL depois de recarregar a página.
8. Edição, mover para a lixeira, restaurar e excluir definitivamente digitando EXCLUIR.
9. No celular: sem rolagem horizontal e com a navegação compacta.
10. Isolamento: logado na Alfa, a URL de um cliente da Beta mostra "não encontrado"; sair e entrar na Beta não mostra nenhum dado da Alfa.
11. Permissões: o leitor não vê ações de escrita e recebe "Sem permissão" ao abrir `/clientes/novo` pela URL; o operador não vê "Excluir definitivamente"; o admin cria um usuário e troca o papel dele.

Regras

- **Guarda global:** todo teste falha se aparecer erro ou aviso no console do navegador, ou qualquer resposta 5xx (seção 21.1).
- Verificação de acessibilidade com `@axe-core/playwright` nas telas principais, sem violações sérias ou críticas.
- Seletores por papel, label e texto (`getByRole`, `getByLabel`), sem depender de classes CSS.
- Sem esperas fixas (`waitForTimeout`): os testes esperam pelo estado da tela.
- Cada teste prepara os próprios dados e roda isolado dos outros.

27. Problemas debatidos na reunião

Os problemas levantados na reunião de devolutiva da parte 1 não estão escritos neste documento, de propósito. Faz parte da avaliação saber se a dupla anotou ou se lembra deles.

- Até o fim do dia 2, a dupla abre uma issue por problema, com a label `reuniao`, descrevendo o problema como entendeu e o critério de aceite que propõe.
- Cada problema é resolvido por um PR vinculado à sua issue (`Closes #N`).
- Os avaliadores comparam as issues com a lista que guardaram e contam separadamente: problemas lembrados, entendidos de forma diferente, esquecidos e resolvidos.
- Dúvidas sobre o que foi dito podem virar issue `question`, mas a lista não é fornecida pelos avaliadores.

28. Entrega e avaliação da parte 2

A nota da parte 2 vai de 0 a 100, mais até 10 pontos de diferenciais, na mesma escala de 0 a 4 da seção 16. Ela só é calculada se o sistema subir com `./start.sh` numa máquina limpa e se o que sobe for a versão compilada.

Entregáveis

- Tag `v2.0.0` na `main`, com CHANGELOG, pelo fluxo de release do Gitflow.
- README atualizado: autenticação, logs e Grafana, seed (com o usuário de acesso), testes unitários e QA, cada um com o comando para rodar.
- Issues da reunião (seção 27) abertas e fechadas por PR.
- Apresentação de 30 minutos: login e rotas protegidas, um erro provocado na interface e encontrado no Grafana, testes unitários e QA rodando ao vivo, e o passeio pelos PRs.

Pesos

Critério	Peso (pontos)	O que é observado
Correções da parte 1	10	Console limpo, build compilado e complemento opcional (seção 21)
Autenticação e JWT	15	Segurança, implementação do JWT, rotas públicas e autenticadas na API e na interface, documentação (seção 22)
Multi tenant e permissões	15	Isolamento entre tenants, papéis e permissões na API e na interface, gestão de usuários (seção 22)

Critério	Peso (pontos)	O que é observado
Testes unitários dos endpoints	15	Os 17 endpoints, casos de 401 e 403, teste de isolamento, cobertura mínima (seção 24)
Automação de QA da interface	10	Cenários, perfis de tela, guarda de console, acessibilidade (seção 26)
Logs centralizados	10	Agente, Loki, Grafana provisionado, redação de dados sensíveis (seção 23)
Seed	5	Dois tenants, cobertura dos casos, determinismo e idempotência (seção 25)
Problemas da reunião	10	Quantos foram lembrados, registrados e resolvidos (seção 27)
Fluxo de trabalho	10	Gitflow, PRs pequenos, revisões reais, issues e release (seção 13)

Diferenciais novos

Somam-se aos da seção 15, dentro do mesmo teto de 10 pontos:

- Detecção de reuso de refresh token, revogando a sessão inteira.
- CI no GitHub Actions rodando testes unitários e QA em todo PR, como check obrigatório para o merge.
- QA também em Firefox e WebKit.
- Alerta no Grafana para taxa de erros 5xx.

Checklist de aceite da parte 2

- [ ] `./start.sh` sobe aplicação, Loki, Alloy e Grafana sem passos manuais
- [ ] Nenhum erro ou aviso no console do navegador em todas as telas
- [ ] Containers rodando a versão compilada
- [ ] Complemento opcional no cadastro e na edição
- [ ] Rotas de clientes respondem 401 sem sessão; registro, login, sessão, renovação e saída funcionando
- [ ] JWT com as claims definidas, validado por schema e sem dado sensível no payload
- [ ] Telas protegidas redirecionam para o login e voltam à rota de origem

- [ ] Nenhum dado de um tenant aparece para outro, na API e na interface
- [ ] Papéis respeitados na API (403) e na interface (ações escondidas e página Sem permissão)
- [ ] Logs dos três containers no Grafana, sem senha nem token
- [ ] Testes unitários dos 17 endpoints e teste de isolamento passando, com a cobertura mínima
- [ ] `./start.sh --seed` cria os dois tenants e pode rodar duas vezes
- [ ] QA da interface passando no desktop e no celular
- [ ] Issues da reunião abertas e fechadas por PR
- [ ] Tag `v2.0.0` na `main`, com CHANGELOG

29. Cronograma sugerido da parte 2

São 13 dias úteis até a v2.0.0. As correções da parte 1 e as issues da reunião vêm primeiro; autenticação e multi tenant vêm antes de testes e QA, porque os dois dependem deles.

Dia	API e infraestrutura	Interface e QA
1	Issues das correções e da reunião; build compilado da API; complemento opcional (migration nova)	Console limpo, error boundary e regra `no-console`
2	Tabelas `tenants`, `users` e `sessions`; migration que leva os clientes da v1 para um tenant padrão; prazo das issues da reunião	Pacote do Playwright com a guarda de console
3	Módulo de tokens JWT, registro, login e sessão	Telas de login e de criar conta
4	Renovação, saída, proteção das rotas e OpenAPI	Rotas protegidas, redirecionamento e renovação no cliente HTTP
5	Filtro por tenant no repositório e unicidade por tenant	Cabeçalho com usuário, tenant e sair
6	Mapa de permissões, checagem por rota e 403	Ações escondidas por permissão e página Sem permissão

Dia	API e infraestrutura	Interface e QA
7	Rotas de usuários e regra do último ADMIN	Tela `/usuarios`
8	Loki, Alloy, Grafana provisionado e redação dos logs	Integração completa
9	Testes unitários de autenticação, JWT, usuários e healthcheck; teste de isolamento	Cenários de QA de autenticação e permissões
10	Testes unitários das rotas de clientes e cobertura mínima	Cenários de QA de clientes, isolamento, celular e acessibilidade
11	Seed com os dois tenants	Problemas da reunião
12	Problemas da reunião e ajustes de revisão	Ajustes de revisão
13	`release/2.0.0`, CHANGELOG, README e tag `v2.0.0`	Ensaio da apresentação

Como na parte 1, os dois devs precisam ter PRs relevantes nas duas camadas: a divisão da tabela é por frente de trabalho, não por pessoa.